



Network Architecture Lab - Semester Project

Project Final Report

Author:
Sheng-Wen Chen [414932]

June 2026

Contents

1	Introduction	3
2	Background	3
2.1	Token-Bucket Policing and TCP Windows	3
2.2	Prior Work in ns-3	4
2.3	What Changes in a Physical Linux Setup	4
3	Experimental Setup	5
3.1	Initial Wehe-Based Attempt	5
3.2	Custom Congestion-Control Module	5
3.3	Handling Kernel Ring-Buffer Overwrite	6
3.4	Final Automation Pipeline	6
3.5	Receiver-Side Window Clamp	7
4	Experiment 1: Sender-Side CWND Capping	7
4.1	Parameter Sweep	7
4.2	Empirical Cap Selection	8
4.3	Early Sanity Check: Large Queue	8
4.4	Sweep Results	9
4.5	Why the Original Formula Overestimated	10
4.6	Remaining Model Mismatch	11
5	Experiment 2: Receiver-Side RWND Control	12
5.1	Motivation	12
5.2	Method	12
5.3	Results	12
6	Experiment 3: Toward Receiver-Side Estimation	13
6.1	Relation to Policing-Rate Inference	13
6.2	Cumulative Arrival at the Receiver	13
6.3	Receiver-Assisted Enqueue-Rate Estimation	13
7	Limitations	13
8	Conclusion	14

1. Introduction

Internet Service Providers often use rate limiters to enforce a maximum traffic rate for a subscriber, an application class, or a traffic aggregate. A common implementation is a Token Bucket Filter (TBF): tokens are generated at a configured rate, packets consume tokens before transmission, and excess packets are either delayed in a queue or dropped when the queue is full. From the user's point of view, this creates a simple question: if the bottleneck rate is known or can be inferred, can a TCP sender use the full allowed rate without repeatedly overflowing the policer and entering loss recovery?

Two previous semester projects studied this question in simulation. Friedli studied congestion-control behavior behind a TBF and proposed fixed-window algorithms that cap the sender's effective window around the bandwidth-delay product (BDP). Rimkevicius studied the complementary problem of estimating the TBF policing rate from packet traces, with the strongest receiver-side result based on the derivative of cumulative byte arrivals. Both works used ns-3, where the bottleneck, queue, loss events, and RTT are controlled and observable.

The goal of this project was to bridge these two works into a physical Linux setup. The central question is not whether the formulas are correct in an ideal simulator, but whether they remain useful when the sender is a real Linux TCP stack, the rate limiter is the Linux `tc` TBF qdisc, packets pass through a NIC driver, and measurements must be extracted from kernel state. The project therefore built a client-server testbed, implemented a Reno-like congestion-control module with a configurable congestion-window cap, added kernel-log based instrumentation, tested receiver-side receive-window clamping, and compared the simulated algorithms against physical measurements. The main result is nuanced. The idea of limiting the effective TCP window is valid in the physical setup: a carefully chosen cap can avoid repeated loss recovery and greatly improve throughput in small-queue cases. However, the original $\text{BDP} + Q - \text{MSS}$ formula often overestimates the correct cap. In the physical setup, the application and TCP stack can enqueue packets into the TBF faster than the TBF drains them. Part of the configured qdisc queue is therefore already consumed by the enqueue/drain mismatch, and cannot be counted as useful extra window. Accounting for this effect explains many of the discrepancies between simulation and the physical experiments.

2. Background

2.1. Token-Bucket Policing and TCP Windows

A TBF is described by three parameters: a token generation rate R_{tbf} , a burst size B , and a queue limit Q_{config} . The token rate determines the long-term output rate. The burst size determines how many tokens can accumulate when the sender is idle or sending below the token rate. The queue limit determines how much traffic can wait for tokens before packets are dropped.

TCP controls sending through two windows. The congestion window (`cwnd`) is chosen by the congestion-control algorithm at the sender. The receive window (`rwnd`) is advertised by the receiver and expresses how much data the receiver is willing to accept. The effective sending window is approximately:

$$W_{\text{send}} = \min(\text{cwnd}, \text{rwnd}).$$

Traditional loss-based algorithms such as TCP Reno and TCP Cubic grow `cwnd` until loss, then reduce it. This behavior is useful when loss indicates shared-network congestion, but it can be inefficient behind a per-flow TBF. If the user has a dedicated rate limit, the best behavior is often to send steadily at the token rate rather than probing above it and recovering from avoidable drops.

2.2. Prior Work in ns-3

Friedli's project, *Revisiting Traffic Congestion Control Protocols at the Last Mile*, studied how TCP New Reno, Cubic, and BBR interact with a TBF in ns-3. The key observation was that the maximum useful amount of in-flight data is related to the BDP and to the bottleneck queue. The project proposed:

$$W_1 = \text{BDP} + Q_{\text{config}} - \text{MSS}$$

and later found that a simpler cap,

$$W_2 = \text{BDP} + \text{MSS},$$

also reaches the TBF rate in many configurations while avoiding buffer build-up. In simulation, the algorithms improved small-buffer cases by roughly 33% for New Reno and 17% for Cubic. For larger queues or larger bursts, default TCP already performs closer to the TBF rate, so the relative gain decreases.

Rimkevicius's project, *Inference of Traffic Differentiation Policing Rate*, studied how to infer R_{tbf} from one traffic trace. It compared sender-side and receiver-side methods. The most accurate method was receiver-side cumulative-arrival differentiation: compute cumulative received bytes over time, differentiate the curve, and use a statistic such as the median or a lower percentile of the derivative samples. Loss-based methods were more conservative, but often underestimated the true policing rate. This project used that work as the natural path toward a receiver-driven system: first infer R_{tbf} , then choose a window cap from the inferred rate.

2.3. What Changes in a Physical Linux Setup

The ns-3 model hides several details that matter in Linux. First, Linux traffic control is organized around queueing disciplines, or qdiscs. The `tc(8)` manual describes a qdisc as the queue into which the kernel places a packet before handing packets to the network-adapter driver, and distinguishes shaping, which controls outgoing transmission, from policing, which applies to arriving traffic [5]. The `tc-tbf(8)` manual describes TBF as a traffic-control qdisc that dequeues packets according to token availability and queues packets up to a configured `limit` while waiting for tokens [6]. This matters for our physical setup: applying a TBF to the server's root qdisc naturally shapes server egress traffic. Shaping traffic after it has arrived at the client is not the normal direct use of TBF and would require redirecting ingress traffic through a virtual interface such as `ifb`. In this project the stable experiments therefore placed the TBF on the server egress path, which emulates the bottleneck seen by server-to-client traffic while keeping the setup reproducible.

Second, the sender does not place one MTU-sized packet at a time into the qdisc. With TCP Segmentation Offload (TSO) and Generic Segmentation Offload (GSO), the kernel or NIC may handle large segments that are later split into MTU-sized packets. This produced large qdisc enqueue events and NIC requeues in early experiments, making the measured `cwnd` behavior difficult to interpret. The final automation disables TSO and GSO before applying the TBF.

Third, measuring `cwnd` at high frequency is itself difficult. Polling with `ss` cannot keep up with fast changes because it requires repeated Netlink dumps. Kernel logging with `pr_info` gives better visibility, but Linux uses a finite ring buffer, so frequent logs are overwritten unless they are extracted and cleared during the transfer.

3. Experimental Setup

3.1. Initial Wehe-Based Attempt

The first implementation attempted to reuse Wehe as a replay source. Wehe is a traffic-differentiation measurement system from Li et al. [4]. Its core technique is record and replay: the authors first record traffic generated by real applications such as YouTube or Netflix, then replay the same packet payloads, sizes, and timings between a mobile client and a Wehe server. Each test compares an original replay, whose payload remains recognizable to deep-packet-inspection devices, with a control replay that preserves the same traffic pattern but bit-inverts the payload to hide the application signature. Wehe then compares the throughput distributions of the original and control replays to decide whether content-based traffic differentiation is present.

However, Wehe was not suitable for the first physical validation step. The replay rate and total transmitted data were not easy to control precisely, and the `ss`-based background process used to extract `cwnd` could not sample fast enough to reveal the expected Reno sawtooth. Figure 1 shows an example of the resulting measurement. The trace is useful as a first sanity check, but it is not precise enough for selecting a window cap.

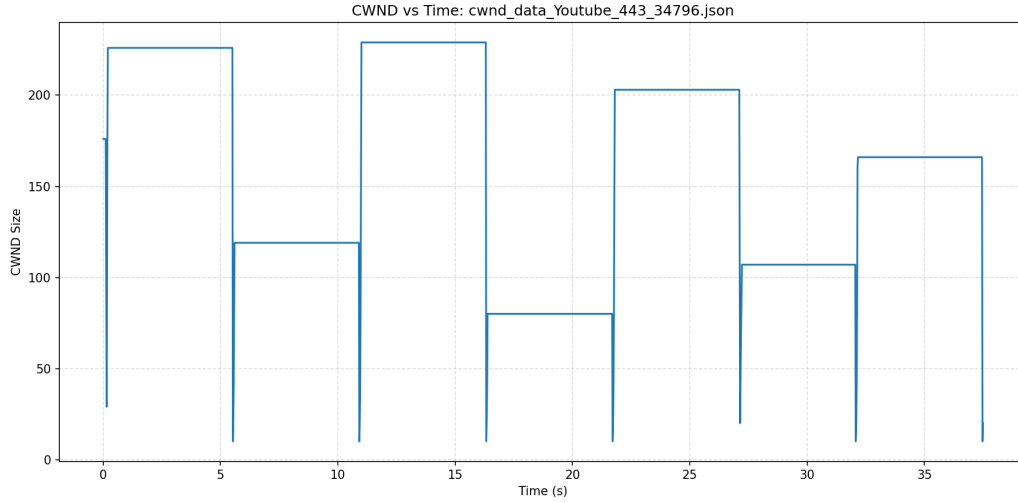


Figure 1: Early Wehe-based experiment. Polling `ss` did not capture the fast `cwnd` transitions needed for this project.

3.2. Custom Congestion-Control Module

The project then moved to a controlled bulk-transfer setup and implemented a custom Linux congestion-control module, `my_cca`. The module is Reno-style: it implements slow start, additive increase, Reno-like slow-start threshold reduction, and an undo callback. The important modification is a module parameter, `cwnd_limit`. When this parameter is nonzero, both slow start and congestion avoidance apply:

$$\text{cwnd} \leftarrow \min(\text{cwnd}, \text{cwnd_limit}).$$

When `cwnd_limit = 0`, the cap is disabled and the module behaves as an uncapped Reno-like baseline. This made it possible to compare capped and uncapped behavior while keeping the same logging and analysis pipeline.

The module logs `cwnd`, RTT samples, bytes acknowledged, bytes sent, and a phase label whenever enough ACKs have passed or the phase changes. The phase labels are:

- `slow_start`, when the sender is below `ssthresh`;

- `congestion_avoidance`, during normal Reno additive increase;
- `fast_retransmit`, when TCP enters recovery after duplicate ACKs or SACK;
- `loss_recovery`, when TCP enters loss or congestion-window reduction.

Logging every ACK was too expensive and quickly overwrote the kernel ring buffer. The final version logs every 100th ACK unless the TCP phase changes. A sequence number was added to detect missing log records.

3.3. Handling Kernel Ring-Buffer Overwrite

The first custom-module runs still produced misleading traces: for example, a 20 GiB transfer did not appear to start from the expected initial `cwnd`: 10. The cause was not TCP behavior, but instrumentation loss. By the time logs were extracted at the end of a transfer, early `pr_info` records had already been overwritten in the **kernel ring buffer**.

In Linux kernel code, `printk()` and its convenience macros such as `pr_info()` are the standard way to write diagnostic messages from inside the kernel. The Linux kernel documentation states that these messages are written to the kernel log buffer, which is a ring buffer exported to user space through `/dev/kmsg`, and that `dmesg` is the usual way to read it [8]. The `dmesg(1)` manual describes `dmesg` as the tool used to examine or control the kernel ring buffer, and notes that `dmesg -clear` clears it [7]. In our module, each selected ACK event calls `pr_info()` with the current `cwnd`, RTT sample, bytes acknowledged, bytes sent, TCP phase, destination IP, and destination port. Because the log buffer has finite capacity, high-frequency `pr_info()` calls can overwrite older records before the user-space analysis script reads them. This is why the early experiments appeared to miss the beginning of slow start.

The final server therefore starts a background log extractor. It repeatedly reads `dmesg`, appends relevant kernel logs to a context file, clears the ring buffer, and sleeps for a configurable interval. This allowed the project to capture the beginning of the transfer and the phase transitions more reliably. Figure 2 shows the type of broken trace observed before this fix.

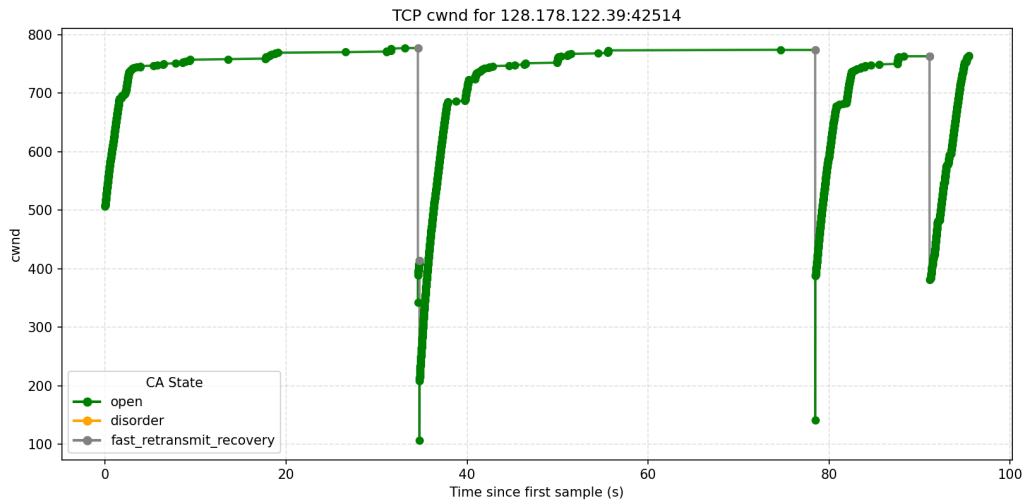


Figure 2: Early custom-CCA trace before reliable ring-buffer extraction. Missing early records made the `cwnd` evolution look inconsistent.

3.4. Final Automation Pipeline

The final experiment driver is `client_server/run.sh`. It performs the following steps for each run:

1. reset the root qdisc on the server interface;

2. install a Linux TBF qdisc with the chosen rate, burst, and queue limit;
3. disable TSO and GSO using `ethtool`;
4. rebuild, sign, unload, and reload `my_cca` with the requested `wnd_limit`;
5. start the Python server with the selected TCP congestion-control algorithm;
6. start the kernel-log extractor and the optional TBF enqueue-rate extractor;
7. wait for one client connection and send the configured amount of data;
8. filter kernel logs for the specific client IP and ephemeral port;
9. generate a `wnd` plot and JSON summaries for throughput, RTT, phase counts, and loss-related events.

The server sends generated bytes in bulk, with `SO_MAX_PACING_RATE` set to 1 Gbit/s by default. Unless otherwise stated, the experiments use 1 GiB transfers, a base RTT around 0.4 ms, and MSS 1460 B. The physical topology is an isolated local network with the server and client in the same subnet. The intended bottleneck is therefore the server egress TBF, not the router or the wider Internet.

3.5. Receiver-Side Window Clamp

The sender-side cap is useful for experimentation, but a realistic client cannot normally install a custom congestion-control module on a remote server. Therefore the project also tested whether the same behavior can be induced from the client by limiting `wnd`. The Python client uses:

$$SO_RCVBUF = TCP_WINDOW_CLAMP = W_{\text{target}}.$$

Since the sender transmits according to $\min(\text{wnd}, \text{rwnd})$, advertising a smaller receive window should constrain the sender even when the server-side `wnd` cap is disabled.

4. Experiment 1: Sender-Side CWND Capping

4.1. Parameter Sweep

The main sweep varied the TBF queue size, burst size, and token generation rate:

- $Q_{\text{config}} \in \{6250, 12500, 25000, 50000, 100000\}$ bytes;
- $B \in \{5840, 12500, 25000, 50000, 100000\}$ bytes;
- $R_{\text{tbf}} \in \{250, 500, 750, 1000\}$ Mbit/s.

For each configuration, four strategies were compared:

- uncapped Reno-like behavior using `my_cca` with `wnd_limit = 0`;
- an empirical cap found by decreasing from the observed Reno top-`wnd` until loss was acceptable;
- Friedli's original $\text{BDP} + Q_{\text{config}} - \text{MSS}$ cap;
- a plain BDP cap, $\lfloor \text{BDP}/\text{MSS} \rfloor$.

The empirical loss threshold was 0.20 for the two smallest queue sizes and 0.05 for larger queues. This change should be kept in mind when comparing empirical choices across queue-size groups.

4.2. Empirical Cap Selection

The empirical cap was not another closed-form formula. It was a measured reference point used to understand what window the physical setup could actually sustain for each TBF configuration. The first step was to run the same configuration with normal Reno-like behavior, using `my_cca` with `cwnd_limit = 0`. From this uncapped run, the analysis extracted the post-slow-start `cwnd` samples before the connection entered a loss state. The useful samples are the top points of the Reno sawtooth: they indicate how far Reno can grow before the TBF and TCP recovery mechanisms push it down again. Averaging these top `cwnd` values gives an initial estimate of the largest safe window for that configuration.

The second step was to rerun the transfer with explicit server-side caps. The search started near the uncapped Reno top-`cwnd` estimate and then gradually decreased the configured `cwnd_limit`. For each capped run, the pipeline recorded throughput and a loss rate computed from the kernel phase labels, mainly the fraction of samples marked as `loss_recovery`. A stricter diagnostic also considered `fast_retransmit`. The empirical cap is therefore the cap found by physical measurement: it is the value that removes most of the loss-recovery behavior while keeping the transfer close to the best observed throughput. This empirical value is not meant to be deployable by itself; it is the ground truth used to judge whether the formulas from the simulation papers remain accurate on the Linux testbed.

4.3. Early Sanity Check: Large Queue

The first stable setup used $R_{\text{tbf}} = 1$ Gbit/s, burst size 1 MB, and $Q_{\text{config}} = 50000$ B. With RTT around 0.4 ms, the BDP is about 50000 B, so the original formula predicts a cap close to 66–67 MSS-sized segments. This experiment is documented in the project notes in `client_server/README` and summarized again in `client_server/report1/optimalCWND.md`. The measured result matched the prediction: `cwnd_limit` around 66–67 produced a flat and stable trace.

However, the throughput improvement over Reno was small: roughly 9.20 s for the capped runs versus about 9.51 s on average for Reno-like runs when sending 1 GiB. This is not a contradiction of the ns-3 work. Friedli’s results already showed that once the TBF queue reaches approximately the BDP, Reno can often keep the outgoing link busy despite its sawtooth. The physical setup first validated this large-queue regime before moving to smaller queues.

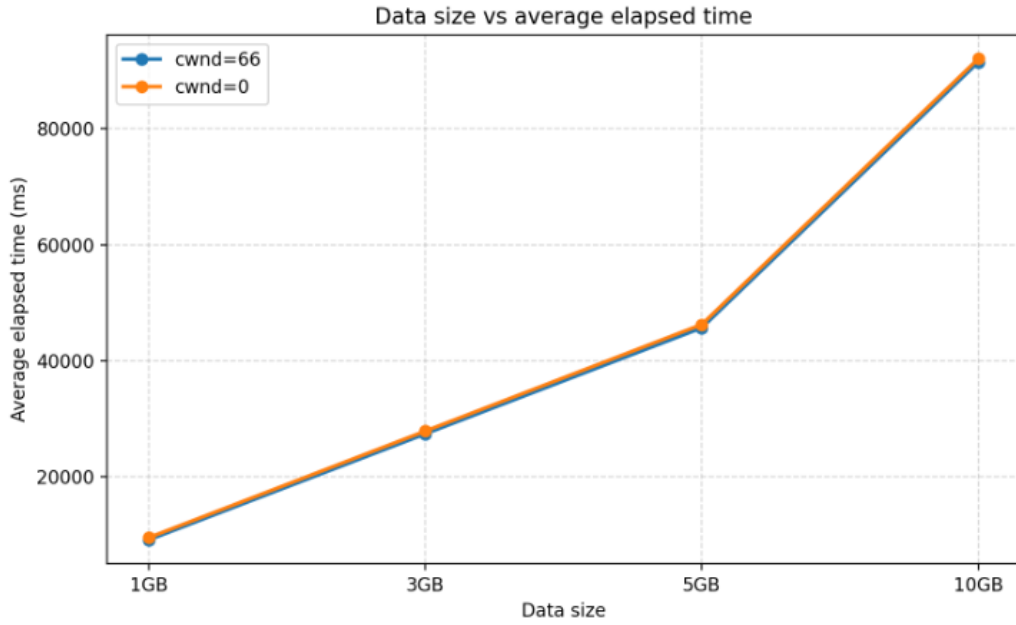


Figure 3: Early large-queue sanity check comparing uncapped Reno-like behavior with a `cwnd` cap near 66 segments.

4.4. Sweep Results

Across all 100 queue/burst/rate points, the empirical cap was the most reliable high-throughput choice. The generated analysis in `client_server/report4` reports the following aggregate metrics:

Strategy	Mean throughput (MiB/s)	Best-throughput wins
TCP Reno	55.92	15
Empirical cap	67.00	60
BDP + $Q_{\text{config}} - \text{MSS}$	55.75	12
BDP	56.13	13

Table 1: Aggregate result over the 100 physical TBF configurations parsed in `report4`.

The gain is concentrated in small queues. Table 2 shows the average empirical-over-Reno throughput gain by queue size.

Queue size (B)	Mean empirical gain over Reno (MiB/s)
6250	36.44
12500	15.39
25000	1.23
50000	0.18
100000	2.15

Table 2: The benefit of a window cap is largest when the TBF queue is small.

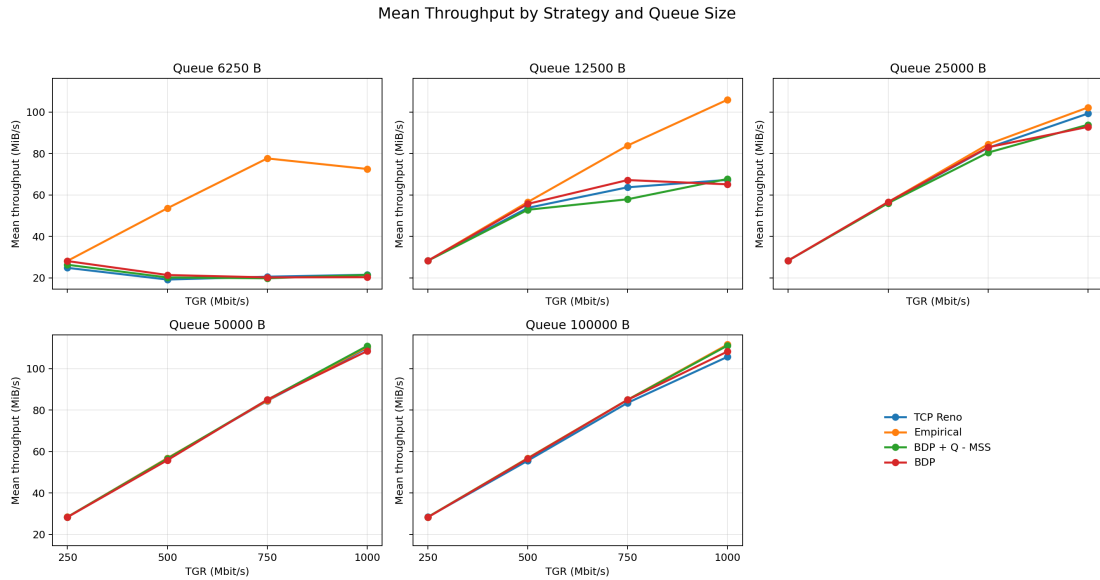


Figure 4: Mean throughput by strategy and queue size. Empirical capping dominates in small queues, while all strategies converge for larger queues.

The largest positive empirical-over-Reno case occurred with $Q_{\text{config}} = 6250$ B, burst size 12500 B, and $R_{\text{tbf}} = 750$ Mbit/s: empirical capping reached 79.46 MiB/s while Reno reached only 10.47 MiB/s. The largest negative case was small: with $Q_{\text{config}} = 25000$ B, burst size 25000 B, and $R_{\text{tbf}} = 1000$ Mbit/s, empirical capping reached 104.69 MiB/s while Reno reached 108.95 MiB/s.

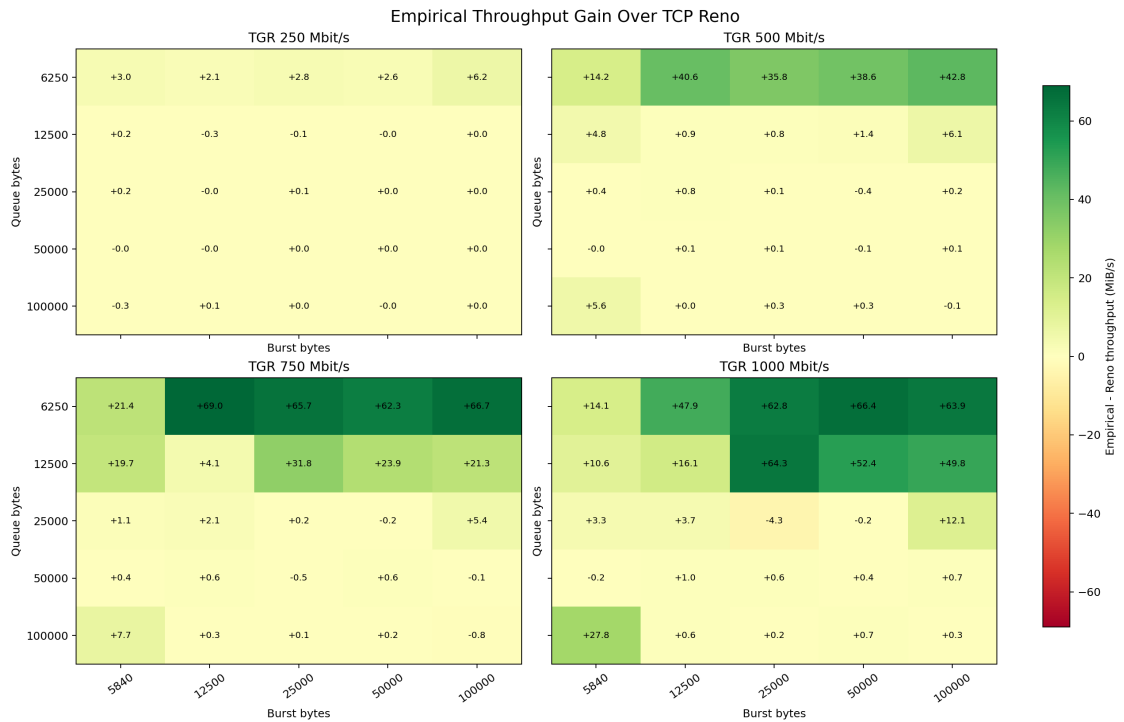


Figure 5: Empirical cap gain over Reno. The high-gain region is concentrated at small queues and high token generation rates.

4.5. Why the Original Formula Overestimated

The original algorithm assumes that the configured TBF queue can be used as extra in-flight capacity:

$$W = \text{BDP} + Q_{\text{config}} - \text{MSS}.$$

This is accurate when the packet arrival rate into the TBF is close to the token generation rate. In the physical setup, however, the server application sends in bulk and the TCP stack can enqueue into the TBF faster than the qdisc drains. Let R_{arrival} be the short-window rate at which bytes arrive at the TBF enqueue path. When $R_{\text{arrival}} > R_{\text{tbf}}$, the queue grows during one RTT by approximately:

$$Q_{\text{tbf}} = \max(0, (R_{\text{arrival}} - R_{\text{tbf}})RTT_{\text{base}}).$$

This queue growth is not free buffer space. It is a burst penalty that must be subtracted from the configured queue:

$$Q_{\text{effective}} = \max(0, Q_{\text{config}} - Q_{\text{tbf}}).$$

The revised estimate is therefore:

$$W = \left\lfloor \frac{\text{BDP} + Q_{\text{effective}} - \text{MSS}}{\text{MSS}} \right\rfloor.$$

When $Q_{\text{tbf}} > Q_{\text{config}}$, even one RTT of enqueue/drain mismatch does not fit into the queue. The project used a conservative reduced-BDP branch:

$$\text{BDP}_{\text{usable}} = \text{BDP} \cdot \min\left(1, \frac{Q_{\text{config}}}{Q_{\text{tbf}}}\right), \quad W = \left\lfloor \frac{\text{BDP}_{\text{usable}} - \text{MSS}}{\text{MSS}} \right\rfloor.$$

This model explains why $\text{BDP} + Q_{\text{config}} - \text{MSS}$ was too aggressive in many small-queue physical runs. For example, with $R_{\text{tbf}} = 500$ Mbit/s, $RTT = 0.4$ ms, and $Q_{\text{config}} = 50000$ B, the BDP is 25000 B. If packets arrive at the TBF near 1 Gbit/s, then the burst penalty is also around 25000 B. The effective queue is therefore about 25000 B, and the expected cap becomes roughly:

$$\left\lfloor \frac{25000 + 25000 - 1460}{1460} \right\rfloor = 33,$$

not 50. This matches the empirical region observed in the physical tests.

4.6. Remaining Model Mismatch

The arrival-aware correction is an improvement, but not a complete model. In the very small queue cases, especially $Q_{\text{config}} = 6250$ B, the reduced-BDP branch can underpredict the best empirical `cwnd`. For example, some 500 Mbit/s and 250 Mbit/s runs were best around `cwnd` 7 and 5 respectively, while the conservative formula predicted smaller values. The reason is that R_{arrival} is not a stable whole-transfer constant. The relevant value is a short burst into the TBF over approximately one base RTT, and this value depends on offloads, pacing, qdisc behavior, and the current TCP phase.

It is important to separate two different problems that looked similar in the early notes. The full kernel ring buffer was a measurement problem: it made the beginning of the `cwnd` trace disappear, so the plots could look as if TCP started from an impossible state. After the background `dmesg` extractor was added, this artifact was greatly reduced. It was not the network mechanism that caused the throughput plateau.

The plateau and collapse in the small-queue experiments came from the data path itself. A bulk-sending server can deliver packets to the TBF enqueue path faster than the TBF token rate. During these short bursts, the configured qdisc queue is partly consumed before it can act as useful BDP-plus-queue headroom. If the cap is too large, this pressure pushes the flow into loss recovery; if the cap is small enough, the sender stays near the TBF rate without repeated recovery. This is why R_{arrival} matters, and why the same R_{tbf} and BDP can lead to different practical caps when the sender pacing and qdisc behavior change.

One possible follow-up would be to modify the server so that R_{arrival} is intentionally controlled. For example, the server could expose `SO_MAX_PACING_RATE` as an experiment parameter, add application-level pacing between send calls, or use `TCP_NOTSENT_LOWAT` to limit how much unsent data accumulates in the TCP stack. This would make it possible to run a cleaner experiment with $R_{\text{arrival}} \approx R_{\text{tbf}}$, and then with known ratios such as $1.25R_{\text{tbf}}$ or $2R_{\text{tbf}}$. The reason this was not the main experiment is that the target deployment should work with an ordinary bulk sender, not only with a cooperative sender that has already been paced to the bottleneck rate. Pacing the server would be useful for isolating the model, but it would also hide exactly the burst pressure that made the physical setup differ from ns-3.

5. Experiment 2: Receiver-Side RWND Control

5.1. Motivation

Directly capping server-side `cwnd` is useful for controlled experiments, but a deployable receiver-side system should not require modifying the sender’s kernel. Since TCP sends according to $\min(\text{cwnd}, \text{rwnd})$, the client can attempt to impose the same effective cap by advertising a small receive window.

5.2. Method

For each selected configuration, the server was run with `my_cca` but with `cwnd_limit = 0`, so the server-side cap was disabled. The client then ran:

```
python3 Client/client.py -rwnd-segments <empirical-cwnd>.
```

The client script converts the segment count to bytes using MSS 1460 B, sets `SO_RCVBUF`, and sets `TCP_WINDOW_CLAMP`. This forces the sender’s effective window to be limited by the advertised receive window.

5.3. Results

The receiver-side cap reproduced the sender-side empirical behavior in most cases. Table 3 shows representative examples from `report3/RWND.md`.

Queue (B)	R_{tbf} (Mbit/s)	Empirical <code>cwnd</code>	Sender cap (MiB/s)	Client <code>rwnd</code> cap (MiB/s)
6250	500	7	56.94	56.23
12500	500	9	56.58	56.60
25000	750	18	84.34	85.27
50000	500	33	56.79	56.75
100000	1000	102	111.98	111.70

Table 3: Representative receiver-side `rwnd` tests. Advertising the empirical window from the client closely matches sender-side `cwnd` capping.

Some small-window cases are sensitive: for example, increasing `rwnd` from 9 to 10 segments in the 750 Mbit/s, 6250 B queue case caused a large drop-rate increase and throughput collapse. This confirms that receiver-side control is powerful enough to reproduce the cap, but also that the selected cap must be accurate.

6. Experiment 3: Toward Receiver-Side Estimation

6.1. Relation to Policing-Rate Inference

The natural final system would combine the two previous papers as follows:

1. the receiver infers R_{tbf} from arrivals, using a method such as cumulative-byte differentiation;
2. the receiver estimates RTT;
3. the receiver computes a target window;
4. the receiver advertises that target through `rwnd`.

The physical experiments support steps 3 and 4, but show that step 1 is not sufficient by itself. Inferring the output policing rate tells us the long-term bottleneck rate. It does not tell us how quickly the sender is enqueueing into the TBF, nor how much of the configured qdisc queue is already consumed by burst pressure.

6.2. Cumulative Arrival at the Receiver

The cumulative-arrival method from Rimkevicius’s work is still useful for estimating the bottleneck rate under balanced conditions. If the flow is already constrained near the token generation rate, the slope of cumulative received bytes approximates R_{tbf} . However, the empirical `rwnd` needed in the physical setup also depends on R_{arrival} . The receiver only observes delivered bytes after the TBF; it does not directly observe how fast the sender tried to enqueue packets before the TBF.

This explains why a receiver can estimate the BDP but still fail to choose the empirical receive window in aggressive bulk-send cases. The hidden variable is not the delivered rate, but the enqueue pressure at the qdisc.

6.3. Receiver-Assisted Enqueue-Rate Estimation

7. Limitations

The project validates the core idea in a real Linux setup, but several limitations remain.

First, the bottleneck was implemented as a Linux server-egress qdisc. Real ISP policing is usually downstream of the sender and may be ingress from the user’s perspective. Linux can emulate ingress shaping through `ifb`, but early tests with `ifb0` did not expose a clean backlog signal, so the stable experiments used server egress.

Second, RTT was very small and difficult to control precisely in the physical LAN. Some experiments attempted to add `netem` delay, but queue placement and interaction between `netem` and TBF changed the results. More complete validation should repeat the sweep across controlled RTTs.

Third, the experiments focus on Reno-like behavior through `my_cca`. Cubic and BBR were studied in ns-3 but not fully reproduced in the physical setup.

Fourth, the empirical cap search used loss thresholds that differed by queue range. This was practical during experimentation, but a final system should define a single objective function balancing throughput, latency, and loss.

Finally, the TBF queue, NIC driver, offload settings, and Linux pacing interact in ways that are hard to reduce to one closed-form formula. Disabling TSO and GSO improved interpretability, but it also means the setup is not exactly the default production Linux network path.

8. Conclusion

References

- [1] Franziska Friedli, *Revisiting Traffic Congestion Control Protocols at the Last Mile*, EPFL NAL Semester Project, 2025.
- [2] Martynas Rimkevicius, *Inference of Traffic Differentiation Policing Rate*, EPFL NAL Semester Project, 2025.
- [3] Tobias Flach et al., *An Internet-Wide Analysis of Traffic Policing*, ACM SIGCOMM, 2016.
- [4] Fangfan Li et al., *A Large-Scale Analysis of Deployed Traffic Differentiation Practices*, ACM SIGCOMM, 2019.
- [5] Linux manual page, *tc(8)*, <https://man7.org/linux/man-pages/man8/tc.8.html>.
- [6] Linux manual page, *tc-tbf(8)*, <https://man7.org/linux/man-pages/man8/tc-tbf.8.html>.
- [7] Linux manual page, *dmesg(1)*, <https://man7.org/linux/man-pages/man1/dmesg.1.html>.
- [8] Linux kernel documentation, *Message logging with printk*, <https://docs.kernel.org/core-api/printk-basics.html>.